

Hybrid Transformer-Based Context-Aware Secret Leakage Detection

for Cloud CI/CD and Infrastructure-as-Code

Tarun Swami
B24046, IIT Mandi
b24046@students.iitmandi.ac.in

September 28, 2026

Abstract

This report presents a local-first security scanner for detecting leaked secrets in cloud infrastructure and CI/CD code. The system combines deterministic regular expressions, entropy analysis, placeholder and documentation filtering, Infrastructure-as-Code (IaC) context, cloud-resource context, CI/CD context, explainable risk scoring, and optional CodeBERT semantic evidence. The deterministic rules engine remains the final security decision layer. The project includes a command-line scanner, JSON/HTML/SARIF reports, a secure ZIP dashboard workflow, GitHub Code Scanning integration, leakage-safe Prowl training, a custom IaC/CI-CD benchmark, and a resumable Transformer training pipeline.

Contents

1	Project Summary	2
1.1	Objectives	2
2	System Architecture	2
2.1	Decision principle	2
3	Detection and Risk Model	3
4	Security and Privacy Controls	3
5	Machine Learning Pipeline	3
6	Evaluation Results	4
6.1	Held-out Prowl result	4
6.2	Custom 600-sample benchmark	4
7	Testing and CI/CD Integration	4
8	Usage Interfaces	4
8.1	CLI	5
8.2	Dashboard	5
9	Limitations and Responsible Deployment	5
10	Conclusion	5

1 Project Summary

The project addresses a practical weakness of binary secret scanners: the same token can have very different operational risk depending on where it appears. A credential-like value in documentation or a test fixture should not receive the same decision as a high-entropy credential inside a production Terraform IAM resource or a deployment workflow.

1.1 Objectives

- Detect common AWS, GitHub, API-key, token, password, and private-key patterns.
- Estimate Shannon entropy and reduce low-information false positives.
- Identify Terraform, CloudFormation, AWS resources, and CI/CD platforms.
- Produce an explainable score from 0 to 100 with SAFE, LOW, MEDIUM, HIGH, and CRITICAL levels.
- Never serialize complete secret values in JSON, HTML, or SARIF output.
- Support local scanning, secure ZIP scanning, GitHub Actions, and optional Transformer inference.

2 System Architecture

Figure 1 shows the principal data path.

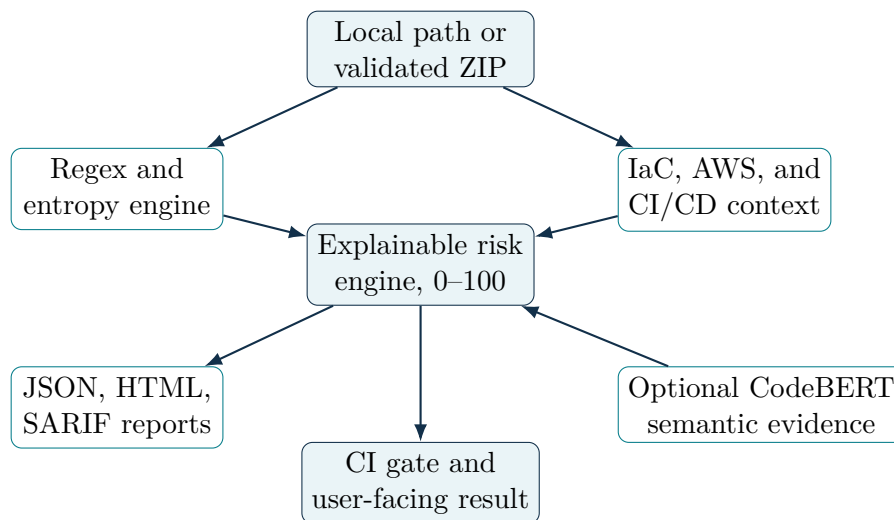


Figure 1: Hybrid context-aware detection architecture.

2.1 Decision principle

The Transformer is supporting semantic evidence. It is not the sole production decision maker because its custom IaC/CI-CD false-positive rate is high. The rules engine explains why a finding is risky and controls the CI exit code.

3 Detection and Risk Model

The scanner discovers supported extensions including Terraform, YAML, JSON, Python, shell, text, Markdown, and environment files. It identifies candidate values, computes Shannon entropy, applies placeholder and documentation reductions, and enriches findings with context.

For a candidate string with character probabilities p_i , entropy is:

$$H(X) = - \sum_i p_i \log_2(p_i).$$

The score is clamped to the interval $[0, 100]$:

$$S = \min(100, \max(0, \sum \text{positive factors} + \sum \text{reductions})).$$

Configured positive factors include secret detection (+40), high entropy (+15), Terraform or CloudFormation context (+10), AWS resource (+10), sensitive resource (+5), CI/CD context (+20), production path (+15), and AWS credential type (+5). Reductions include placeholder value (-30), test/example file (-20), low entropy (-10), and low confidence (-5).

Score	Level
0–19	SAFE
20–39	LOW
40–59	MEDIUM
60–79	HIGH
80–100	CRITICAL

4 Security and Privacy Controls

- ZIP uploads enforce 100 MB compressed, 500 MB extracted, and 10,000-entry limits.
- Path traversal and symlink entries are rejected.
- Unsupported files are filtered before extraction.
- Temporary extraction directories are deleted after scanning.
- Reports redact complete secret values.
- GitHub Actions runs on the runner and does not send repository code to an external scanner service.
- The public benchmark uses safe example markers rather than live credentials.

5 Machine Learning Pipeline

The optional semantic model uses CodeBERT with an eight-dimensional context vector: entropy, IaC presence, CI/CD presence, AWS presence, secret keyword presence, placeholder signal, normalized content length, and high-entropy flag. The context vector is projected and fused with the CodeBERT representation before classification.

The Prowl corpus is cleaned, deduplicated by SHA-256 of normalized text, balanced, and split without hash overlap. The verified production experiment used 15,000 training samples, 4,000 validation samples, and 4,000 test samples.

Training supports CUDA, FP16, checkpointing, early stopping, explicit subset sizes, sequence length, data-loader workers, and model-only continuation. The laptop fast experiment used 10,000/2,000/2,000 samples, two epochs, batch size 8, and sequence length 96.

6 Evaluation Results

6.1 Held-out Prowl result

Metric	Value
Accuracy	0.9670
Precision	0.9647
Recall	0.9695
F1	0.9671
ROC-AUC	0.9943
False-positive rate	0.0355
False-negative rate	0.0305

6.2 Custom 600-sample benchmark

Method	Accuracy	Precision	Recall	F1	FP
Regex only	0.7833	0.6750	1.0000	0.8060	130
Context-aware rules	1.0000	1.0000	1.0000	1.0000	0
Transformer only	0.6517	0.5648	0.9852	0.7179	205
Hybrid	0.6583	0.5684	1.0000	0.7248	205

The perfect rules score is specific to this synthetic benchmark and must not be generalized to all production repositories. The high Transformer false-positive count supports keeping deterministic context rules as the final decision layer.

7 Testing and CI/CD Integration

The verified local suite contains 17 tests covering entropy, placeholders, secret patterns, IaC analysis, resource extraction, risk scoring, secure ZIP extraction, cleanup, SARIF redaction, and fixture expectations. The repository workflow runs tests, scans product code while excluding intentional test/benchmark/report directories, uploads SARIF to GitHub Code Scanning, and stores JSON/HTML/SARIF artifacts.

External repositories can consume the reusable workflow with:

```
name: Secret scan
on:
  push:
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
  security-events: write
jobs:
  scan:
    uses: swamitarun/Secret-Leakage-Detection-for-Cloud-CI-CD/.github/
      workflows/security-scan.yml@master
    secrets: inherit
```

8 Usage Interfaces

8.1 CLI

```
python scanner/cli.py /path/to/project --fail-on HIGH
python scanner/cli.py . --exclude tests --exclude dataset --exclude
reports
```

8.2 Dashboard

```
streamlit run dashboard/app.py
```

The dashboard provides ZIP upload, secure extraction, scan execution, risk findings, and JSON/HTML downloads.

9 Limitations and Responsible Deployment

The project is a research prototype, not a guarantee of universal secret detection. Organization-specific calibration, unseen provider testing, authentication, HTTPS, rate limiting, background jobs, and privacy controls are required before accepting arbitrary private repositories through a hosted service. Users must rotate any real credential detected by the scanner and investigate access logs.

10 Conclusion

The project delivers an explainable, local-first security workflow for cloud CI/CD and IaC. Its strongest production design is the deterministic context-aware engine integrated into GitHub Actions and SARIF, with the Transformer retained as optional semantic evidence. The implementation is reproducible on a laptop CPU for scanning and on an RTX 2050 for compact training experiments.